

Week-2 [day 1]

Home Assignment

1) File Upload

Incorrect prompt: Generate test cases for file upload.

Response:

When designing test cases for a file upload feature, you need to cover functionality, boundaries, security, and edge cases to ensure robustness.

1. Functional & Standard Uploads

- Valid Single File: Upload a valid file format (e.g., .pdf, .png, .jpg) within the allowed size limit and verify success.
- Drag and Drop: Drag a valid file directly onto the upload zone and verify it registers correctly.
- Multiple Files: Upload multiple valid files at once if multi-file upload is supported.
- File Overwrite/Duplicate: Upload a file with the same name as an existing file and check if it prompts for replacement or renames automatically.
- Cancel/Remove: Cancel an ongoing upload or delete a file after uploading but before form submission.

2. File Size & Format Boundaries

- Maximum Size Limit: Upload a file exactly at the maximum allowed size limit (should pass) and 1 KB above the limit (should fail with an error).
- Zero KB File: Upload an empty (0 KB) file and verify an appropriate error message appears.
- Unsupported Extension: Upload an unpermitted file format (e.g., uploading .exe or .sh when only images are allowed).

- File Extension Case Sensitivity: Upload files with uppercase extensions (e.g., .JPG, .PDF) to verify they are handled correctly.

3. Security & Validation

- MIME-Type Spoofing: Rename an executable file to an image extension (e.g., changing malware.exe to malware.png) and verify server-side validation catches it.
- Malicious Files/Scripts: Attempt to upload files containing embedded scripts (e.g., SVG files with JavaScript or PHP shells) to test for execution prevention.
- File Name Injection: Upload a file with special characters or path traversal sequences in its name (e.g., ../../etc/passwd or file;name.png).
- Long File Name: Upload a file with an extremely long file name (e.g., >255 characters) to ensure system stability.

4. UI, Performance & Network Interruptions

- Progress Bar: Verify the progress bar accurately reflects upload status and completion percentage.
- Network Loss Mid-Upload: Disconnect the network connection halfway through a large upload to verify graceful failure handling.
- Page Refresh: Refresh or navigate away from the page during an upload and ensure appropriate warnings appear.
- Slow Network Conditions: Test uploading on a slow or throttled 3G network to ensure the connection doesn't time out prematurely.

ICEPOT Framework prompt:

context

test the file upload feature for a course management application

instruction

[mandatory] test the file upload functionality

[critical] generate both positive and negative test cases

[priority] generate exactly 10 test cases

[mandatory] test supported file types: PDF, DOC, DOCX, XLS, XLSX, JPG, JPEG, and PNG

[mandatory] test unsupported file types: EXE, BAT, SH, JS, HTML, and ZIP

[mandatory] maximum allowed file size is 10 MB

[mandatory] test files below, equal to, and above the 10 MB limit

[mandatory] test empty/0 KB files

[mandatory] test duplicate file uploads

[mandatory] test multiple file uploads if supported

[mandatory] test upload cancellation

[mandatory] test successful upload and verify the file is available after upload

[mandatory] test upload failure and verify an appropriate error message is displayed

[critical] include security scenarios such as attempting to upload executable, script, HTML, or potentially malicious files

[critical] verify unsupported or potentially malicious files are rejected and not executed

[mandatory] verify appropriate validation/error messages for invalid files

[mandatory] include a mix of positive and negative test cases

[mandatory] include Test Case ID, Priority, Type, Description, Preconditions, Steps, Expected Output, and Test Data

[priority] use P0, P1, and P2 priorities appropriately

[critical] generate exactly 10 test cases and do not exceed 10

test data

supported files: PDF, DOC, DOCX, XLS, XLSX, JPG, JPEG, PNG

unsupported files: EXE, BAT, SH, JS, HTML, ZIP

maximum file size: 10 MB

boundary files: 9.9 MB, 10 MB, 10.1 MB

empty file: 0 KB

security test files: executable/script/HTML files

success criteria

- Valid supported files within the 10 MB limit upload successfully.
- Uploaded files are correctly displayed/available after upload.
- Invalid file types are rejected with a clear error message.
- Files exceeding 10 MB are rejected with a clear size-limit message.
- Empty files are handled according to the defined validation rule.
- Duplicate files are handled according to the application rule.
- Potentially malicious or executable files are rejected and never executed.
- Upload failures provide a clear and user-friendly error message.

failure criteria

- Unsupported or malicious files are uploaded successfully.
- Files larger than 10 MB are accepted.
- Invalid files are accepted without validation.
- Upload fails without an appropriate error message.
- A successfully uploaded file cannot be accessed or displayed correctly.

examples

testcases id | priority | type | description | preconditions | steps | expected output | test data

persona

act as a test lead and review the generated test cases

ensure the test cases are complete, clear, executable, and testable

ensure positive and negative scenarios are balanced

ensure boundary-value scenarios are covered

ensure security-related file upload scenarios are covered

remove duplicate or redundant test cases

ensure exactly 10 test cases are generated

output

generate the test cases in a table

download the test cases as an Excel file

Excel columns:

Test Case ID | Priority | Type | Description | Preconditions | Steps | Expected Output | Test Data | Review Comments

tone

professional and concise

Response:

The Excel file containing the comprehensive test cases is there in this folder name:
Course_Management_File_Upload_Test_Cases

Test Lead Review & Summary

As requested, I have reviewed the test suite to ensure completeness, strict boundary-value coverage, strong security validation, and strict adherence to the 10-test-case constraint.

Review Summary:

- Total Test Cases Generated: Exactly 10
- Priority Distribution: P0 (5), P1 (4), P2 (1)
- Coverage:
 - Supported File Types: PDF, DOC, DOCX, XLS, XLSX, JPG, JPEG, PNG
 - Unsupported File Types: EXE, BAT, SH, JS, HTML, ZIP
 - Boundary Conditions: 0 KB (empty), 9.9 MB (below limit), 10.0 MB (at limit), 10.1 MB (above limit)
 - Security & Vulnerabilities: Executable restriction (EXE, BAT, SH), XSS/Injection restriction (JS, HTML)

- Usability & Edge Cases: Upload cancellation, multiple file upload, duplicate file handling, post-upload verification
-

2) API Testing

Incorrect prompt: Write API test cases for user registration.

Response:

Here is a comprehensive set of API test cases for a User Registration endpoint (POST /api/v1/users/register).

1. Happy Path (Positive Scenarios)

ID	Test Scenario	Request Payload Highlights	Expected Status	Expected Response / Verification
TC-01	Register with all valid mandatory fields	Email, Password, First/Last Name	201 Created	User ID returned, password hashed in DB.
TC-02	Register with mandatory + optional fields	Include phone, avatar_url, bio	201 Created	All provided fields stored correctly.
TC-03	Email normalization (case sensitivity)	TestUser@Domain.com	201 Created	Stored as lowercase testuser@domain.com .

2. Validation & Boundary Scenarios (Negative Scenarios)

ID	Test Scenario	Request Payload Highlights	Expected Status	Expected Response / Verification
TC-04	Missing required field	Omit password field	400 Bad Request	Error message identifying missing parameter.

TC-05	Invalid email format	user@domain, @domain.com, user.com	400 Bad Request	Field validation error for email.
TC-06	Weak password	12345 (fails complexity rules)	400 Bad Request	Error listing missing requirements (length/symbols).
TC-07	Duplicate email registration	Re-send payload from TC-01	409 Conflict	Unique constraint error message.
TC-08	Max length validation	First Name > 255 characters	400 Bad Request	String length limit error.

3. Edge Cases & Security Scenarios

ID	Test Scenario	Request Payload Highlights	Expected Status	Expected Response / Verification
TC-09	Empty request body	{ }	400 Bad Request	Root payload validation error.
TC-10	Incorrect Content-Type header	Send JSON payload with text/plain	415 Unsupported Media Type	Request rejected before body processing.
TC-11	SQL Injection attempt	email: "" OR '1'='1"	400 Bad Request	Input sanitized, no syntax/DB error thrown.
TC-12	XSS Script Injection attempt	first_name: "<script>alert(1)</script>"	201 Created / 400	Input escaped/sanitized in database response.

TC-13	Mass Assignment attempt	Include isAdmin: true or role: "admin"	201 Created	User created with default role (user), elevated privileges ignored.
TC-14	Rate limiting / Brute force	Send 50 registration requests in 10 sec	429 Too Many Requests	Throttling header returned (Retry-After).

ICEPOT Framework prompt:

context

test the User Registration API

instruction

[mandatory] test the user registration API functionality

[critical] generate both positive and negative API test cases

[priority] generate exactly 10 test cases

[mandatory] use HTTP POST method for user registration

[mandatory] API endpoint: /api/users/register

[mandatory] request Content-Type must be application/json

request structure

```
{
  "name": "Test User",
  "email": "testuser@example.com",
  "password": "Test@123",
  "confirmPassword": "Test@123"
}
```

response structure for successful registration

```
{  
  "message": "User registered successfully",  
  "userId": "12345"  
  
}
```

success status code

201 Created

negative status codes to validate

400 Bad Request - invalid or missing input

409 Conflict - email already registered

415 Unsupported Media Type - invalid Content-Type

422 Unprocessable Entity - validation failure

500 Internal Server Error - unexpected server error

test scenarios

[mandatory] test successful registration with valid data

[mandatory] test registration with missing required fields

[mandatory] test invalid email format

[mandatory] test already registered email

[mandatory] test weak/invalid password

[mandatory] test password and confirmPassword mismatch

[mandatory] test empty or null values

[mandatory] test invalid Content-Type

[mandatory] test duplicate registration request

[mandatory] test unexpected server/error response

[critical] validate HTTP method, endpoint, request headers, request body, response body, status code, and response headers where applicable

[mandatory] verify that sensitive information such as passwords is not returned in the response

[mandatory] verify appropriate error messages for negative scenarios

[mandatory] verify that a user is not created when registration fails

[critical] include both positive and negative test cases

[critical] generate exactly 10 test cases and do not exceed 10

API validation criteria

- Valid registration returns 201 Created.

- Response contains the expected success message and user identifier.

- Invalid requests return the appropriate 4xx status code.

- Duplicate email registration returns 409 Conflict.
- Invalid Content-Type returns 415 Unsupported Media Type.
- Validation errors return 400 or 422 according to the API specification.
- Password and other sensitive data must not be exposed in the response.
- Failed registration must not create a duplicate or invalid user.

tools

Postman for manual API testing

RestAssured with Java for API automation

Use Postman request/response validation concepts and RestAssured-compatible test scenarios.

examples

test case id | priority | type | method | endpoint | request | expected status code | expected response | description

persona

act as a Test Lead and API Test Engineer

review each test case for completeness, correctness, clarity, and testability

ensure HTTP method, endpoint, headers, request body, response body, and status codes are validated

ensure positive and negative scenarios are covered

remove redundant test cases

ensure exactly 10 test cases are generated

output

generate the API test cases in a table

download the test cases as an Excel file

Excel columns:

Test Case ID | Priority | Type | HTTP Method | Endpoint | Preconditions | Request Headers | Request Body | Steps | Expected Status Code | Expected Response | Test Data | Review Comments

tone

professional and concise

Response:

The Excel file containing the comprehensive test cases is there in this folder name:

User_Registration_API_Test_Cases

User Registration API Test Cases (/api/users/register)

Role: Test Lead & API Quality Engineer

Method: POST | **Content-Type:** application/json | **Total Test Cases:** 10

Summary of API Validation Rules & Criteria Verified

1. **Positive Registration:** Standard registration returning HTTP 201 Created with userId and confirmation message.
2. **Missing Input Validation:** Missing required field email returning HTTP 400 Bad Request.
3. **Email Format Integrity:** Invalid email format returning HTTP 422 Unprocessable Entity.
4. **Duplicate Prevention:** Existing email registration returning HTTP 409 Conflict.
5. **Password Complexity:** Weak password failing complexity policy returning HTTP 422 Unprocessable Entity.
6. **Password Confirmation:** Mismatch between password and confirmPassword returning HTTP 400 Bad Request.
7. **Empty/Null Payload:** Null and whitespace input fields returning HTTP 400 Bad Request.
8. **MIME-Type Strictness:** Invalid Content-Type header (e.g. text/plain) returning HTTP 415 Unsupported Media Type.
9. **Concurrent Duplicate:** Race condition prevention yielding HTTP 409 Conflict.
10. **System Resilience & Security:** Database connection failure yielding HTTP 500 Internal Server Error without exposing passwords or stack traces.

RestAssured Automation Implementation snippet

```
Java
import io.restassured.RestAssured;
import io.restassured.http.ContentType;
```

```

import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;
import static io.restassured.RestAssured.*;
import static org.hamcrest.Matchers.*;

public class UserRegistrationApiTest {

    @BeforeAll
    public static void setup() {
        RestAssured.baseURI = "https://api.example.com";
    }

    @Test
    public void testSuccessfulRegistration_TC001() {
        String requestBody = "{\n" +
            "  \"name\": \"Test User\",\n" +
            "  \"email\": \"testuser\" + System.currentTimeMillis() + "@example.com\",\n" +
            "  \"password\": \"Test@123\",\n" +
            "  \"confirmPassword\": \"Test@123\"\n" +
            "}";

        given()
            .contentType(ContentType.JSON)
            .body(requestBody)
            .when()
            .post("/api/users/register")
            .then()
            .statusCode(201)
            .body("message", equalTo("User registered successfully"))
            .body("userId", notNullValue())
            .body("$", not(hasKey("password"))); // Ensures password is never returned
    }
}

```

3) Role-Based Access Control

Incorrect prompt: Write test cases for user roles.

Response:

To design comprehensive test cases for **Role-Based Access Control (RBAC)**, tests are typically split into three primary categories: **Positive Testing** (authorized access), **Negative Testing** (unauthorized access/privilege escalation), and **Boundary/Edge Testing** (role changes, session behavior).

Below is a structured test matrix using a common system setup (Admin, Manager, Standard User, Guest).

Test Case Matrix

ID	Module / Feature	Role	Test Scenario	Preconditions	Test Steps	Expected Result	Type
TC-RBAC-01	User Management	Admin	Verify Admin can create a new user and assign roles	Admin logged in	1. Go to Admin Dashboard. 2. Click "Add User". 3. Fill details & assign "Manager" role. 4. Save.	User created successfully; role saved as "Manager".	Positive
TC-RBAC-02	User Management	Standard User	Verify Standard User cannot access Admin Dashboard	User logged in	1. Enter direct Admin URL (/admin/dashboard). 2. Press Enter.	System blocks access, redirects to /403 Forbidden or	Negative

						home page.	
TC-RB AC -03	Reports / Data	Manager	Verify Manager can view and export team reports	Manager logged in	1. Navigate to Reports module. 2. Click "Export CSV".	Reports generate and download successfully for assigned team only.	Positive
TC-RB AC -04	Reports / Data	Manager	Verify Manager cannot view other teams' data	Manager logged in	1. Try accessing another team's report via direct URL (/reports?team_id=999).	System returns "Access Denied" or empty data set scoped to manager's team.	Negative
TC-RB AC -05	Billing / Settings	Manager	Verify Manager cannot modify billing/organization settings	Manager logged in	1. Navigate to Settings. 2. Check for Billing tab.	Billing tab is hidden or disabled for Manager role.	Negative
TC-RB	Profile /	Standard User	Verify Standard User can	User logged in	1. Go to My Profile.	Profile updates	Positive

AC-06	Account		update own profile details		2. Change phone number. 3. Save.	successfully.	
TC-RB AC-07	Profile / Account	Standard User	Verify Standard User cannot edit another user's profile	User logged in	1. Inspect API request for profile edit. 2. Swap user ID to another user's ID. 3. Send request.	System rejects request with HTTP 403 / 401 response.	Security / API
TC-RB AC-08	Public Content	Guest	Verify Guest can access public pages only	Guest (unauthenticated)	1. Navigate to home page, landing page, public docs.	Pages load correctly with login/register buttons visible.	Positive
TC-RB AC-09	Application Access	Guest	Verify Guest is prompted to log in when reaching protected pages	Guest (unauthenticated)	1. Enter protected URL (/app/dashboard).	System redirects guest to /login page.	Boundary

TC-RB AC -10	Session Management	All Roles	Verify immediate access update when user role is modified by Admin	User logged in as Standard User	1. Admin updates User's role to "Manager" in another session. 2. User refreshes page or clicks Manager-only feature.	System updates permissions instantly or forces token refresh to grant access.	Dynamic Role Change
---------------------	--------------------	------------------	--	---------------------------------	---	---	---------------------

Core Areas to Validate

- **UI Visibility:** Buttons, menus, and actions restricted to specific roles must be completely hidden (not just disabled) for unauthorized users.
- **API / Backend Security:** Direct API calls (via Postman, cURL, or Developer Tools) using lower-privilege tokens must return HTTP 403 Forbidden.
- **Privilege Escalation:**
 - **Vertical:** Can a Standard User perform Admin actions by manipulating parameters/payloads?
 - **Horizontal:** Can User A access or edit User B's resources within the same role level?
- **Session Termination:** De-activating or changing a user's role should invalidate active authorization tokens immediately.

ICEPOT Framework prompt:

context

test the user role and access control feature for a course management application

instruction

[mandatory] test user roles and role-based access control (RBAC)

[critical] generate both positive and negative test cases

[priority] generate exactly 10 test cases

[mandatory] test the following roles:

- Admin
- Manager
- Viewer

application domain

Course Management System

permission matrix

Admin:

- Create, view, edit, and delete courses
- Manage users
- Assign and change user roles
- View reports
- Manage application settings

Manager:

- Create, view, and edit courses
- View course reports
- Cannot delete courses
- Cannot manage users or assign roles
- Cannot modify application settings

Viewer:

- View courses

- View course details
- Cannot create, edit, or delete courses
- Cannot manage users
- Cannot view restricted management reports
- Cannot modify application settings

test scenarios

- [mandatory] verify Admin can access all authorized functions
- [mandatory] verify Manager can access only Manager-authorized functions
- [mandatory] verify Viewer can access only Viewer-authorized functions
- [mandatory] verify unauthorized users cannot access restricted pages
- [mandatory] verify Manager cannot perform Admin-only actions
- [mandatory] verify Viewer cannot perform Manager/Admin actions
- [mandatory] verify direct URL access to unauthorized pages is blocked
- [mandatory] verify unauthorized API/action requests are rejected
- [mandatory] verify role changes correctly update user permissions
- [mandatory] verify logged-out users cannot access authenticated role-based pages

- [critical] cover both authorized and unauthorized access scenarios
- [mandatory] verify appropriate error behavior such as 401 Unauthorized or 403 Forbidden
- [mandatory] verify the UI does not expose actions that the user's role cannot perform
- [critical] ensure exactly 10 test cases and do not exceed 10

permission validation

- Admin has full access according to the permission matrix.
- Manager has only Manager permissions.
- Viewer has only Viewer permissions.
- Unauthorized page access is blocked.
- Unauthorized actions are rejected.

- Direct URL access cannot bypass role restrictions.
- API/action-level authorization cannot be bypassed by modifying requests.
- Role changes take effect correctly.
- Unauthenticated users cannot access protected functionality.

examples

test case id | priority | role | type | description | preconditions | steps | expected output

persona

act as a Test Lead specializing in security and role-based access control

review the test cases for completeness, clarity, correctness, and testability

verify that every test case matches the permission matrix

ensure positive and negative authorization scenarios are covered

ensure unauthorized access and privilege-escalation scenarios are included

remove redundant test cases

ensure exactly 10 test cases are generated

output

generate the test cases in a table

download the test cases as an Excel file

Excel columns:

Test Case ID | Priority | Role | Type | Description | Preconditions | Steps | Expected Output |
Test Data | Review Comments

tone

professional and concise

Response:

The Excel file containing the comprehensive test cases is there in this folder name:
Course_Management_RBAC_Test_Cases

Executive Test Summary & Permission Matrix Review

As a **Test Lead specializing in Security and Role-Based Access Control (RBAC)**, I have developed, validated, and reviewed a set of **exactly 10 test cases** for the Course Management Application.

Permission Matrix Overview

Feature / Action	Admin	Manager	Viewer	Unauthenticated
Create / Edit Courses	Allowed	Allowed	Denied	Denied (401)
Delete Courses	Allowed	Denied	Denied	Denied (401)
View Courses & Details	Allowed	Allowed	Allowed	Denied (401)
View Course Reports	Allowed	Allowed	Denied (Restricted)	Denied (401)
Manage Users & Assign Roles	Allowed	Denied	Denied	Denied (401)
Modify Application Settings	Allowed	Denied	Denied	Denied (401)
Direct URL & API Enforcement	Enforced	Enforced	Enforced	Enforced

Highlights of the Excel Workbook Features:

- **Tab 1: Executive Summary:** Contains a high-level Role Permission Matrix and metrics breakdown (Total, Priority, Positive vs. Negative counts).
 - **Tab 2: RBAC Test Cases:** Formatted using a **Dark Navy** palette, explicit column widths, text wrapping, and freeze header rows.
 - **Data Validation & Formatting:** Ready for execution tracking in manual/automated QA pipelines.
-

